1. Aim

To develop a vehicle registration number validator using Python string operations and slicing, without using regular expressions, and to normalise and format valid registration numbers.

2. Problem Explanation

Vehicle registration numbers may be entered in different formats, such as 'tn 09 ab 1234', 'TN-09-AB-1234' and 'Tn09Ab1234'. The program converts these entries into one standard form and checks whether they follow the required pattern.

A valid registration number contains:

- Two letters representing the state.
- Two digits representing the district.
- One or two letters representing the series.
- Four digits representing the vehicle number.

After removing spaces and hyphens and converting letters to uppercase, the valid length is either 9 or 10 characters.

3. Predictions Before Execution

The following six inputs are selected before running the program.

Input	Normalised form	Prediction
tn 09 ab 1234	TN09AB1234	Valid
KA-01-M-0007	KA01M0007	Valid
Tn09Ab1234	TN09AB1234	Valid
TN9AB1234	TN9AB1234	Invalid
TN 22 1234 AB	TN221234AB	Invalid
MH 12 XY 9876	MH12XY9876	Valid

4. Python Program – Normalising, Validating and Formatting

```
def normalise(text):
    """Remove spaces and hyphens and convert to uppercase."""
    return text.strip().replace(" ", "").replace("-", "").upper()


def is_valid(reg):
    """Check state, district, series and vehicle number."""
    if len(reg) not in (9, 10):
        return False

    if not reg[0:2].isalpha():
        return False

    if not reg[2:4].isdigit():
        return False
```

```
    if not (reg[4:-4].isalpha() and len(reg[4:-4]) in (1, 2)):
        return False

    if not reg[-4:].isdigit():
        return False

    return True


def format_number(reg):
    """Format a valid registration number for a receipt."""
    if not is_valid(reg):
        return "INVALID"
    return reg[:2] + " " + reg[2:4] + " " + reg[4:-4] + " " + reg[-4:]


samples = [
    "tn 09 ab 1234",
    "KA-01-M-0007",
    "Tn09Ab1234",
    "TN9AB1234",
    "TN 22 1234 AB",
    "MH 12 XY 9876"
]

for raw in samples:
    reg = normalise(raw)
    status = "valid" if is_valid(reg) else "INVALID"
    print(f"{raw:15} -> {reg:11} {status}")
    if is_valid(reg):
        print("Receipt:", format_number(reg))
```

5. Expected Output

```
tn 09 ab 1234   -> TN09AB1234  valid
Receipt: TN 09 AB 1234
KA-01-M-0007    -> KA01M0007   valid
Receipt: KA 01 M 0007
Tn09Ab1234      -> TN09AB1234  valid
Receipt: TN 09 AB 1234
TN9AB1234       -> TN9AB1234   INVALID
TN 22 1234 AB   -> TN221234AB  INVALID
MH 12 XY 9876   -> MH12XY9876  valid
Receipt: MH 12 XY 9876
```

6. Explanation of String Operations

Operation	**Purpose**
strip()	Removes leading and trailing whitespace.
replace()	Removes spaces and hyphens from the input.
upper()	Converts lowercase letters to uppercase.
len()	Checks whether the normalised number has 9 or 10 characters.
isalpha()	Checks that a section contains only letters.
isdigit()	Checks that a section contains only digits.
Slicing	Separates the state, district, series and final four digits.

7. String Immutability

Strings in Python are immutable, which means their characters cannot be changed after the string is created. The upper() method does not modify the original string. Instead, it returns a new string containing uppercase characters. To keep the changed value, it must be assigned to a variable, for example text = text.upper().

8. Explanation of Slicing

The expression s[-4:] returns the last four characters of the string. The expression s[4:-4] returns the characters beginning at index 4 and ending just before the last four characters.

Example	s[-4:]	s[4:-4]
TN09M1234 (9 characters)	1234	M
TN09AB1234 (10 characters)	1234	AB

These slices depend on the exact string. For the valid 9-character example TN09AB123, the final four characters are B123, so it does not satisfy the required registration format. A valid 9-character registration such as TN09M1234 is actually 10 characters; valid examples with one series letter have 9 characters, such as TN09M1234? No: counting carefully, two state letters + two district digits + one series letter + four digits gives 9 characters. Therefore TN09M1234 is 9 characters, and its s[-4:] is 1234 while s[4:-4] is M.

9. Alternative Concept – Regular Expressions

The same validation can be performed using the re module and a regular expression. The pattern checks the complete string from beginning to end.

```
import re

def is_valid_regex(reg):
    pattern = r"[A-Z]{2}[0-9]{2}[A-Z]{1,2}[0-9]{4}"
    return re.fullmatch(pattern, reg) is not None
```

The regular expression is shorter and clearly expresses the required pattern. The slicing method uses basic string operations and may be easier for beginners to understand step by step. Both methods scan a number of characters proportional to the input length.

10. Time and Space Complexity Analysis

Method	Time complexity	Space complexity	Explanation
String operations and slicing	O(L)	O(L)	The input is scanned and new normalised or sliced strings may be created.
Regular expression	O(L)	O(L) worst-case auxiliary space	The pattern checks the input characters; matching may use internal working memory.

Here, L is the length of the registration number. Since vehicle numbers are short and have a fixed maximum length, both methods are efficient for this task.

11. Execution Evidence

Run the program using the six test inputs and add a screenshot of the output. Check that the four valid and invalid examples match the predictions. Include the receipt format output for valid registration numbers.

- Screenshot showing normalisation of spaces and hyphens.
- Screenshot showing valid and invalid results.
- Screenshot showing the formatted receipt number.

12. Conclusion

The vehicle registration number validator was implemented using Python string methods and slicing without regular expressions. The program removes spaces and hyphens, converts the input to uppercase, checks each section and formats valid numbers for receipts. String immutability and slicing were also examined. The slicing-based method is suitable for this assignment because it clearly demonstrates basic string operations.